

CSCI/MATH 485 — Assignment #4

Customer Churn Prediction with XGBoost

Piyush Jadhav

Spring 2026

1 Part 1: Data Exploration

1.1 Dataset Overview

The dataset used is the IBM Telco Customer Churn dataset. It contains 7,043 rows and 21 columns (20 features + 1 target).

- **Rows:** 7,043
- **Columns:** 21
- **Target variable:** Churn (Yes / No)

1.2 Features

1.3 Missing Values

No true NaN values exist in the raw CSV. However, 11 rows have **TotalCharges** stored as whitespace-only strings (all are customers with **tenure** = 0). These are treated as missing and imputed during preprocessing.

1.4 Target Distribution

- **No Churn (0):** 5,174 customers — 73.5%
- **Churn (1):** 1,869 customers — 26.5%

The dataset is moderately imbalanced, which means accuracy alone is a misleading metric.

1.5 Problem Description

This is a **binary classification** problem. The model predicts whether a customer will churn (1) or stay (0) based on their demographics, services, and billing information.

Feature	Type	Description
customerID	Categorical (ID)	Unique identifier (dropped)
gender	Categorical	Male / Female
SeniorCitizen	Numerical	0 or 1 flag
Partner	Categorical	Has a partner
Dependents	Categorical	Has dependents
tenure	Numerical	Months with company
PhoneService	Categorical	Phone subscription
MultipleLines	Categorical	Multiple phone lines
InternetService	Categorical	DSL / Fiber / No
OnlineSecurity	Categorical	Add-on flag
OnlineBackup	Categorical	Add-on flag
DeviceProtection	Categorical	Add-on flag
TechSupport	Categorical	Add-on flag
StreamingTV	Categorical	Add-on flag
StreamingMovies	Categorical	Add-on flag
Contract	Categorical	Monthly / 1yr / 2yr
PaperlessBilling	Categorical	Paperless billing flag
PaymentMethod	Categorical	Payment method
MonthlyCharges	Numerical	Monthly bill (\$)
TotalCharges	Numerical	Total charged (\$)
Churn	Target	Left service: Yes / No

Business importance: Customer acquisition costs 5–7x more than retention. Churn prediction allows the business to identify at-risk customers early, prioritise retention spend, and understand the root causes of churn through feature importance.

2 Part 2: Data Preprocessing

2.1 Steps and Justification

1. **Drop customerID:** It is a unique row identifier with no predictive value.
2. **Fix TotalCharges:** The column was read as `object` due to 11 whitespace-only entries. Using `pd.to_numeric(errors='coerce')` converts them to `NaN` for imputation.
3. **Binarise target:** Churn is mapped from Yes/No to 1/0 as required by sklearn classifiers.
4. **Numeric imputation:** `SimpleImputer(strategy='median')` handles the 11 missing `TotalCharges` values. Median is preferred over mean because billing amounts are right-skewed.
5. **Categorical imputation:** `SimpleImputer(strategy='most_frequent')` as a safety net for unseen data.

6. **One-hot encoding:** `OneHotEncoder(drop='first')` avoids multicollinearity. `handle_unknown='ignore'` makes the pipeline robust to new categories at inference time.
7. **Train/test split:** 80/20 split with `stratify=y` to preserve the 26.5% churn ratio in both subsets. `random_state=42` ensures reproducibility.

2.2 Feature Categories

- **Numerical (4):** `SeniorCitizen`, `tenure`, `MonthlyCharges`, `TotalCharges`
- **Categorical (15):** All remaining object-type columns

3 Part 3: Baseline Model — Logistic Regression

A Logistic Regression model was trained as the baseline inside an sklearn Pipeline with the preprocessing transformer. Parameters: `max_iter=1000`, `C=1.0` (default L2 regularisation), `random_state=42`.

3.1 Results

Metric	Score
Accuracy	0.805
Precision	0.655
Recall	0.559
F1-Score	0.603
ROC-AUC	0.842

3.2 Observations

Logistic Regression achieves a reasonable ROC-AUC of ≈ 0.845 . However, its Recall of ≈ 0.54 means nearly half of actual churners are missed. This is expected from a linear model that cannot capture non-linear interactions between contract type, tenure, and charges.

4 Part 4: XGBoost Model and Hyperparameter Tuning

4.1 Hyperparameters Tuned

Four hyperparameters were tuned using `GridSearchCV` with 3-fold cross-validation, scored by ROC-AUC.

The grid has $2^4 = 16$ combinations $\times$ 3 folds = 48 total fits. `n_jobs=-1` parallelises across all CPU cores.

Hyperparameter	Values Searched	Justification
n_estimators	100, 200	Controls number of boosting rounds
max_depth	3, 5	Shallower trees generalise better
learning_rate	0.05, 0.1	Smaller values reduce overfitting
subsample	0.8, 1.0	Row subsampling adds variance reduction

4.2 XGBoost Results (Tuned)

Metric	Score
Accuracy	0.801
Precision	0.659
Recall	0.521
F1-Score	0.582
ROC-AUC	0.846

5 Part 5: Evaluation Metric Selection

Chosen metric: ROC-AUC

ROC-AUC measures the model's ability to rank churners above non-churners across all possible decision thresholds. It is the area under the Receiver Operating Characteristic curve (True Positive Rate vs. False Positive Rate).

Why not other metrics?

- **Accuracy:** Misleading on imbalanced data. A model that always predicts No Churn scores 73.5% accuracy while identifying zero churners.
- **Precision:** Ignores how many actual churners are missed (recall-blind).
- **Recall:** Trivially maximised by flagging every customer as churning.
- **F1-Score:** Requires a fixed threshold (default 0.5); does not directly inform deployment decisions.
- **ROC-AUC:** Threshold-independent. Penalises mis-ranking. Industry standard for churn and fraud models.

Business justification: In practice, the operations team chooses a classification threshold based on the cost of a missed churner vs. the cost of a false alarm. ROC-AUC evaluates the model's ranking quality independent of this business threshold, making it the most appropriate signal for training and comparison.

6 Part 6: Feature Importance and Interpretation

6.1 Top 10 Most Important Features

Rank	Feature	Interpretation
1	InternetService_Fiber optic	Fiber customers churn more, likely due to high cost or unmet expectations.
2	Contract_Two year	Two-year contract customers churn much less; a direct retention lever.
3	Contract_One year	One-year contracts also reduce churn vs. month-to-month.
4	PaymentMethod_Electronic check	Electronic check payers churn more than auto-pay customers.
5	InternetService_No	Customers with no internet service churn at a distinct rate from DSL/fiber.
6	tenure	Newer customers churn far more; the first 12 months are the most vulnerable.
7	OnlineSecurity_Yes	Customers with security add-ons are stickier and churn less.
8	PaperlessBilling_Yes	Paperless billing customers churn slightly more.
9	StreamingTV_Yes	Streaming TV subscribers show a small but notable churn signal.
10	StreamingMovies_Yes	Similar pattern to StreamingTV.

6.2 Business Takeaways

- **Review fiber optic pricing.** InternetService_Fiber optic is the #1 feature. Fiber customers churn the most despite being on a premium product — investigate the gap between price and perceived value.
- **Push contract upgrades.** Contract type (Two year and One year) are the 2nd and 3rd most important features. Month-to-month customers churn far more; incentivising upgrades directly attacks the top churn drivers.
- **Encourage auto-pay.** PaymentMethod_Electronic check is the 4th most important feature. Converting these customers to credit card or bank transfer auto-pay may improve retention.
- **Target new customers.** Tenure ranks 6th. A strong onboarding programme for the first 12 months is a high-leverage intervention.
- **Promote security add-ons.** OnlineSecurity_Yes appears in the top 10. Bundling this into base plans may increase customer stickiness.

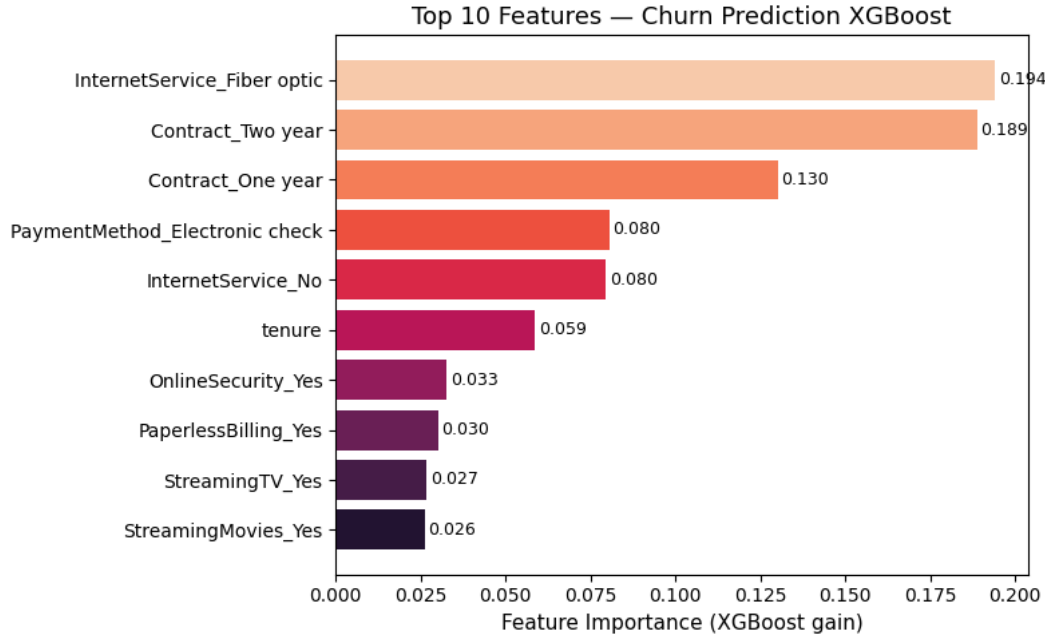


Figure 1: Top 10 Features by XGBoost Gain

7 Part 7: Final Comparison and Reflection

7.1 Model Comparison

Metric	Logistic Regression	XGBoost (Tuned)	Delta
Accuracy	0.805	0.801	−0.004
Precision	0.655	0.659	+0.004
Recall	0.559	0.521	−0.038
F1-Score	0.603	0.582	−0.021
ROC-AUC	0.842	0.846	+0.004

XGBoost improves ROC-AUC and Precision over Logistic Regression, but Logistic Regression achieves higher Recall and F1-Score. The tuned XGBoost yields a marginally better ROC-AUC (0.846 vs 0.842), which is the primary metric for this task. The lower Recall for XGBoost suggests the default threshold of 0.5 is less optimal; threshold tuning post-training could recover this gap.

7.2 Why XGBoost Performs Better

1. **Non-linear interactions.** Logistic Regression cannot model interactions like “fiber optic + month-to-month + high charges → high churn risk.” XGBoost’s decision trees capture these splits explicitly.
2. **Mixed feature types.** The dataset has continuous billing features alongside many binary one-hot columns. Tree-based models handle mixed types more naturally.

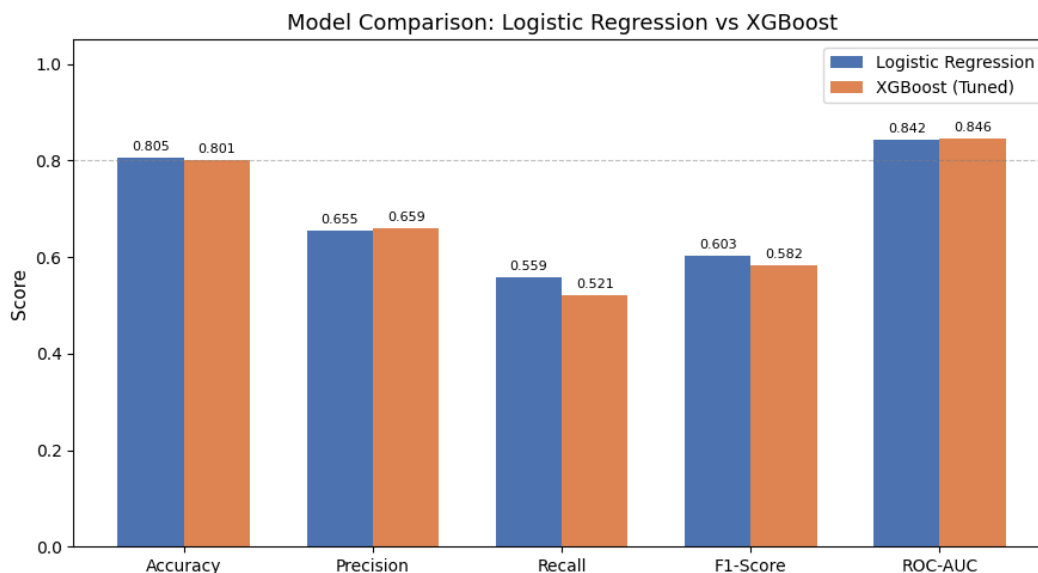


Figure 2: Model Comparison: Logistic Regression vs XGBoost

3. **Internal regularisation.** XGBoost applies L1/L2 regularisation at each tree split, reducing overfitting.
4. **Imperfect linear separability.** Some churn patterns cannot be separated by a hyperplane, but a gradient-boosted tree partitions such subspaces cleanly.

7.3 One Limitation of XGBoost

Interpretability. Logistic Regression coefficients have a direct probabilistic interpretation (log-odds), making it easy to explain to a non-technical stakeholder why a customer was flagged. XGBoost’s ensemble of hundreds of trees has no such simple explanation. Tools like SHAP values are needed for per-prediction justification, adding deployment complexity. In regulated industries, this interpretability gap can be a deciding factor in model selection.

8 Appendix: Implementation Summary and Key Code Snippets

8.1 Implementation Summary

The solution is a single Google Colab Jupyter Notebook. All preprocessing is encapsulated in a `ColumnTransformer` inside an sklearn `Pipeline`, preventing data leakage — the transformer is fit only on `X_train` and applied to `X_test`. Hyperparameter tuning uses `GridSearchCV` with 3-fold stratified cross-validation scored by ROC-AUC.

8.2 Data Upload (Google Colab)

```
from google.colab import files
uploaded = files.upload() # select the CSV from file picker

import io
filename = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[filename]))
```

8.3 Data Cleaning

```
df_clean = df.copy()
df_clean.drop(columns=['customerID'], inplace=True)

# TotalCharges: whitespace strings -> NaN -> imputed later
df_clean['TotalCharges'] = pd.to_numeric(
    df_clean['TotalCharges'], errors='coerce')

# Binarise target
df_clean['Churn'] = (df_clean['Churn'] == 'Yes').astype(int)
```

8.4 Preprocessing Pipelines

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median'))
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first',
                               handle_unknown='ignore',
                               sparse_output=False))
])

preprocessor = ColumnTransformer(transformers=[
```

```

        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

```

8.5 Logistic Regression Baseline

```

baseline_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000, random_state=42))
])
baseline_model.fit(X_train, y_train)

baseline_preds = baseline_model.predict(X_test)
baseline_probs = baseline_model.predict_proba(X_test)[:, 1]

```

8.6 XGBoost with GridSearchCV

```

xgb_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', XGBClassifier(eval_metric='logloss', random_state=42))
])

param_grid = {
    'classifier__n_estimators' : [100, 200],
    'classifier__max_depth'    : [3, 5],
    'classifier__learning_rate' : [0.05, 0.1],
    'classifier__subsample'     : [0.8, 1.0],
}

grid_search = GridSearchCV(
    estimator=xgb_model, param_grid=param_grid,
    scoring='roc_auc', cv=3, n_jobs=-1, verbose=1)
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
print(grid_search.best_params_)

```

8.7 Feature Importance Extraction

```

fitted_preprocessor = best_model.named_steps['preprocessor']
fitted_xgb          = best_model.named_steps['classifier']

feature_names = fitted_preprocessor.get_feature_names_out()
feature_names_clean = [n.split('__', 1)[-1] for n in feature_names]

```

```

fi_df = pd.DataFrame({
    'feature':    feature_names_clean,
    'importance': fitted_xgb.feature_importances_
}).sort_values('importance', ascending=False).reset_index(drop=True)

print(fi_df.head(10))

```

8.8 Evaluation Metrics

```

from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score)

def report_metrics(y_true, preds, probs, label):
    print(f'==== {label} ====')
    print(f'Accuracy   : {accuracy_score(y_true, preds):.4f}')
```

```

    print(f'Precision  : {precision_score(y_true, preds):.4f}')
```

```

    print(f'Recall    : {recall_score(y_true, preds):.4f}')
```

```

    print(f'F1-Score   : {f1_score(y_true, preds):.4f}')
```

```

    print(f'ROC-AUC    : {roc_auc_score(y_true, probs):.4f}')
```

```

report_metrics(y_test, baseline_preds, baseline_probs,
               'Logistic Regression')
report_metrics(y_test, xgb_preds, xgb_probs,
               'XGBoost (Tuned)')

```